03 · Handshake · validar Type 2, Type 3 e obter a sessão

IMPLEMENTADO até HandshakeSession · integração no vídeo ainda por desenvolver

PROXY · continua ProxyHandshake.perform()

HandshakeWire.read(SERVER_HELLO) devolve frame2.
Separa prefixo NB do ciphertext + tag.
HandshakeKeySchedule.serverHelloProtection()
→ mesma chave/IV que o servidor derivou
HandshakeCrypto.decrypt(AAD = header2 || NB || H1)
Verifica tag GCM antes de devolver plaintext.

SERVIDOR · continua ServerHandshake.perform()

Type 2 já foi enviado.
Tem Z, PSK, NP, SRV e os dois frames.
HandshakeCrypto.sha256(frame1, frame2) → H12
HandshakeKeySchedule.sessionSecret()
PSK + Z + NP + SRV + H12 → segredo da sessão
Apaga a cópia do array Z com fill(0).

4 · Aceitar resposta e calcular segredo

HandshakePayloadCodec.decodeServerHello()
Valida SEQY = SEQX+1 e CCC = proposta.
DiffieHellmanSession.calculateSecret(B) → Z
HandshakeCrypto.sha256(frame1, frame2) → H12
HandshakeKeySchedule.sessionSecret() → segredo
Apaga a cópia do array Z com fill(0).

4 · Aguardar confirmação do proxy

HandshakeWire.read(CLIENT_FINISHED)
→ ProtocolMessageCodec.read()
HandshakeKeySchedule.clientFinishedProtection()
segredo + H12 → chave/IV próprios de Type 3
Não começa a transmitir media nesta fase.

5 · Construir e cifrar ClientFinished

ClientFinishedPayload(SEQX+2, SHA256(SRV), media)
HandshakePayloadCodec.encodeClientFinished()
HandshakeKeySchedule.clientFinishedProtection()
HandshakeWire.frame() → header3
HandshakeCrypto.encrypt(AAD = header3 || H12)
HandshakeWire.send() → ProtocolMessageCodec.write()

5 · Verificar a confirmação recebida

HandshakeCrypto.decrypt(AAD = header3 || H12)
HandshakePayloadCodec.decodeClientFinished()
Valida SEQZ = SEQX+2.
MessageDigest.isEqual(R, SHA256(SRV))
HandshakeWire.validateMedia(media)
Autenticação GCM confirma posse do segredo DH.

TYPE 3 · header || GCM(SEQZ || SHA256(SRV) || MEDIA) →

6 · Em ambos os participantes: construir HandshakeSession

HandshakeCrypto.sha256(frame1, frame2, frame3) → H123 (transcript completo / sessionId)
HandshakeKeySchedule.trafficKeys(segredo, H123) → SessionKeys(proxyToServer, serverToProxy)
Cada direção tem ProtectionKeys(key, iv); KeyMaterial guarda cópias e oculta segredos em toString().
HandshakeSession(keys, media, firstMediaSequence = SEQX+3, sessionId = H123)
HandshakeDemo imprime apenas media / ID público, fecha o socket e termina o teste.

Confirmação vista por cada lado

Servidor: Type 3 verificado → aceita a sessão. Proxy: Type 2 verificado + Type 3 enviado.
O proxy só terá confirmação da aceitação pelo servidor ao receber um primeiro registo protegido válido.

PRÓXIMA INTEGRAÇÃO · percurso pretendido, ainda sem implementação

Browser → pedido HTTP ao proxy → handshake com o servidor → Type 4 (blocos protegidos) → proxy
Proxy verifica sequência/tag, decifra e entrega bytes MP4 ao browser → Type 5 (END autenticado).
Falta definir/implementar contadores de nonce, registos Type 4/5, Range/catálogo e métricas.
O IV de tráfego derivado aqui é uma base: não se pode reutilizá-lo diretamente em todos os blocos.